



Projeto Aplicativo - Last Dodo

Ana Luisa Mendes Câmara, Danielle Caroline Soares Costa e Silva , Isabella Andrade Caldeira

Instituto de Ciências Exatas – Universidade de Brasília (UnB)

Departamento de Ciência da Computação – Universidade de Brasília (UnB)

261039938@aluno.unb.br, 261013619@aluno.unb.br, 261017108@aluno.unb.br

Resumo. *Este artigo expõe o desenvolvimento e a concretização de Last Dodo, uma releitura de Journey of the Prairie King, foi desenvolvida e codificada inteiramente na linguagem assembly RISC-V. Afastando-se da estética original, o projeto trás uma proposta temática própria substituindo o protagonista por um pássaro dodô, cuja mecânica consiste em esquivar-se de animais inimigos para sobreviver. A criação de jogos em ambientes de hardware de baixo nível impõe restrições severas, como gerenciamento de memória e sincronização do loop de renderização gráfica. Executado no simulador RARS, o projeto implementa recursos avançados de sistemas computacionais, como Bitmap Display, tratamento de entrada via teclado por MMIO, detecção de colisões em tempo real por matrizes lógicas e síntese de áudio por chamadas de sistemas MIDI.*

Palavras-chave: *Assembly RISC-V. RARS. Bitmap Display. Double Buffering. Otimização de Jogos.2.*

1. Introdução

O jogo *Journey of The Prairie King* (Jornada do Rei da Pradaria) é um minigame incluso no jogo de simulação e RPG *Stardew Valley*, criado pelo desenvolvedor solo americano Eric "ConcernedApe" Barone, e publicado em parceria com o estúdio britânico Chucklefish. Dentro do jogo, os usuários podem, além de explorar e cuidar de sua fazenda, jogar minigames, entre eles *Journey of The Prairie King*. Este especificamente está inicialmente disponível para ser jogado na máquina de arcade do Saloon Fruta Estelar, uma das construções na vila. Após vencer o jogo no salão, também é possível colocar um arcade e jogar na casa do próprio jogador.

A proposta do projeto aplicativo foi fazer uma releitura de *Journey of The Prairie King*, inteiramente na linguagem de programação Assembly RISC-V, a ser montado e rodado pelo software RARS (*RISC-V Assembler and Runtime Simulator*). O grupo tomou algumas liberdades criativas, substituindo o protagonista por um pássaro dodô, o último de sua espécie, que precisa se esquivar de animais inimigos para continuar vivo.

2. Metodologia

O projeto foi implementado utilizando arquivos do tipo .s para código fonte, e .data para sprites de personagens, mapas e itens do cenário. Todo o código foi escrito na linguagem Assembly RISC-V, e o jogo pode ser rodado por completo após a montagem, dentro do próprio RARS.

2.1 Tela do jogo

A tela do jogo funciona a base da ferramenta Bitmap Display do RARS, que mostra uma imagem – neste caso o frame atual do jogo, com mapa, personagens e o HUD (*Head Up Display*) – a partir de uma matriz de 320x240 pixels. Cada um deles ocupa um espaço de 8 bits, no qual ficam armazenadas as informações sobre a cor daquele pixel, no formato BBGGGRRR, ou seja, dois bits para o azul (B), três para o verde (G) e três para o vermelho (R).

A atualização da tela do jogo é feita por um sistema de *double buffering*, no qual um dos quadros (*frames*) é mostrado enquanto o próximo está sendo carregado pixel por pixel na memória, sem aparecer na tela. Isso nos causou sérios problemas de jogabilidade, pois a resposta do jogo possui um certo atraso. Ao jogar o jogo, isso fica perceptível principalmente na movimentação do dodô e dos inimigos, já que ao dar um passo, o deslocamento só é mostrado quando acontece a troca de quadro.

Os elementos que povoam a tela do jogo são dispostos em camadas, sendo o cenário e obstáculos a camada inferior, com os personagens dispostos por cima, e por fim o HUD, com informações sobre a fase atual e o número de vidas restantes, representado por corações.



2.2 Comandos do teclado

Os comandos do teclado são capturados pela ferramenta *Keyboard and Display MMIO (Memory-Mapped Input/Output) Simulator*, e então processados pelo programa. Dessa forma, é possível fazer com que o dodô movimente-se pelo mapa e se esquive dos inimigos, utilizando um sistema WASD – tecla W desloca o dodô um passo para cima, A desloca o dodô um passo para a esquerda, S desloca o dodô um passo para baixo e D desloca o dodô um passo para a direita.

O grupo tinha intenção de implementar também um sistema de ataque à distância (arremesso de penas como projéteis) e de curto alcance (bicada), a ser acionado por teclas pressionadas no teclado, mas as tentativas resultaram em um grande número de problemas técnicos e bugs, principalmente de renderização. Entre eles aconteceu o sumiço e reaparição frequentes de personagens na tela, e um grave atraso na resposta do programa, que tornava o jogo impossível de jogar.

2.3 Cenário

Os cenários foram estruturados a partir de matrizes salvas no formato .data, utilizando de um código numérico para definir o que seria renderizado em cada espaço do mapa – espaços vazios, grama, tocos de madeira, árvores, etc. Essas matrizes são então interpretadas pelo código de renderização, encaixando os sprites – também salvos como matrizes em arquivos .data, com as cores de cada pixel – nos espaços definidos na matriz do mapa. Depois disso, os pixels são mostrados pelo Bitmap Display, nas cores definidas pelo fundo ou pelos sprites.

2.4 Personagens

Os sprites dos personagens seguem a mesma lógica de arquivo e renderização dos objetos do cenário, com a adição de movimentos. A movimentação segue lógicas diferentes para os tipos de inimigos, e a direção do movimento determina em que posição o sprite será mostrado no frame que está renderizando.

Quanto à movimentação, o inimigo Rato da fase 1 possui um padrão de movimento simples que independe do jogador: ele anda em uma direção fixa e, ao ler a matriz do mapa e detectar um obstáculo ou borda, inverte sua direção, basicamente indo e voltando em linha reta de um obstáculo ao outro. Já o Cachorro da fase 2 segue uma lógica de perseguição direta. Ele calcula a distância (X e Y) em relação à posição do jogador e tenta reduzir essa distância, dando prioridade à direção na qual está mais longe, e sempre checar a matriz do mapa para não ignorar a colisão.

3. Resultados Obtidos

A seguir é apresentada a avaliação do desempenho e do comportamento sistêmico do jogo *Last Dodo* após a implementação das otimizações de baixo nível vistas na metodologia.

3.1 Eficiência de Renderização e Estabilidade de Aplicação

A implementação conjunta do algoritmo de restauração local (Radar) e do mecanismo de *Double Buffering* foi capaz de reduzir de forma significativa o custo de renderização gráfica durante a execução do jogo. Inicialmente, a atualização completa da tela de **320 × 240 pixels** necessitava da reescrita de todos os pixels da memória de vídeo a cada ciclo, aumentando o tempo necessário para gerar cada quadro.

Com a implementação do Radar, apenas a região mais próxima do personagem passou a ser atualizada, diminuindo consideravelmente a quantidade de acessos à memória e tornando a execução mais ágil. E o uso do Double Buffering eliminou o efeito de cintilação e de quebra de imagem, pois a troca entre quadros ocorre apenas depois da conclusão da renderização do frame na memória secundária.

3.2 Balanceamento da Jogabilidade

Durante o desenvolvimento do jogo também foi necessário ajustar a movimentação dos inimigos para manter a jogabilidade equilibrada. Como o processamento do Assembly requer que todas as operações sejam executadas pelo processador, uma atualização simultânea dos inimigos tornou a experiência de jogo bastante difícil.

Para solucionar esse problema, foi implementado um sistema de controle baseado em turnos, no qual o cachorro realiza um movimento a cada movimento do jogador, porém travado ao chegar em obstáculos. Com essa estratégia a velocidade do inimigo foi reduzida tornando a partida mais equilibrada sem comprometer o desempenho geral da aplicação.

3.3 Gerenciamento dos Estados do Jogo

O funcionamento do jogo foi organizado por meio de diferentes estados de execução, responsáveis por controlar situações com a partida em andamento, colisões entre os personagens e o encerramento da partida.

Quando acontece uma colisão entre o Dodô e algum dos inimigos ou obstáculo, uma rotina de inicialização reposiciona os personagens em suas posições iniciais e atualiza variáveis necessárias para a continuidade do jogo, que seja necessário reiniciar toda a aplicação.

Quando as vidas se esgotam o jogo entra em estado de Game Over, interrompendo a partida e exibindo a tela correspondente ao encerramento. Essa

organização torna possível controlar de forma consistente as transições entre os diferentes estados do jogo.

4. Conclusão

O desenvolvimento do jogo *Last Dodo* em linguagem Assembly RISC-V atingiu com sucesso alguns dos objetivos propostos para o projeto prático da disciplina. No entanto, foi a percepção compartilhada pelo grupo que a linguagem Assembly e o ambiente e ferramentas do RARS possuem limitações inatas para o desenvolvimento de jogos, causando diversos problemas e atrasos tanto na experiência desenvolvimento quanto na jogabilidade.

Por causa dessas limitações e problemas enfrentados, não foi possível implementar muitas das mecânicas presentes no minigame original, e nem uma quantidade satisfatória de mapas e inimigos. sistema de power-ups, sistema de vitória ou pontuação (luta contra um chefe ou histórico de melhores pontuações), possibilidade de atacar os inimigos em curta ou longa distância, mecânica de pause, tela de início e tela de fim de jogo apropriada.

Por fim, concluímos que foi uma experiência de muito aprendizado, porém com muitos momentos de dificuldade e estresse. Não consideramos que atingimos um nível de desenvolvimento adequado para uma releitura fiel do jogo, porém finalizamos o jogo sem erros de montagem e com gráficos, sistema de vidas e movimentação aceitáveis.

5. Referências Bibliográficas

- STARDEW Valley. Em: WIKIPEDIA: the free encyclopedia. Flórida: Wikimedia Foundation, 2026. Disponível em: https://en.wikipedia.org/wiki/Stardew_Valley. Acesso em: 16 jul. 2026.
- STARDEW Valley. Em: STARDEW Valley Wiki. [S. l.]: ConcernedApe, 2026. Disponível em: https://pt.stardewvalleywiki.com/Stardew_Valley. Acesso em: 16 jul. 2026.
- ARCADE do Rei da Pradaria. Em: STARDEW Valley Wiki. [S. l.]: ConcernedApe, 2026. Disponível em: [https://pt.stardewvalleywiki.com/Arcade do Rei da Pradaria](https://pt.stardewvalleywiki.com/Arcade_do_Rei_da_Pradaria). Acesso em: 16 jul. 2026.
- RARS: RISC-V Assembler and Runtime Simulator. Versão modificada disponibilizada pelo professor. Manual de ajuda integrado. Disponível no Aprender 3.